Event Driven Updates with Apollo Subscriptions

Course Overview

Course Overview

Hi, everyone. My name is Jonathan Mills, and welcome to my course, Event Driven Updates with Apollo Subscriptions. I'm a cloud application architect at World Wide Technology, and with modern web applications, keeping what's on the screen up to date with changes happening elsewhere in the application can be a bit of a challenge. And in this course, we're going to talk about one way to solve that problem by using GraphQL and Apollo subscriptions to update the data on the client side in real time as changes are made on the back end. Some of the major topics we're going to cover include building out a subscription API on the back end, so we'll build the back end, consuming that subscription on the front end, securing that subscription using a token-based authentication, so we're going to secure the API, and then filtering subscriptions by adding some variables to it. Now by the end of this course, you'll know everything you need to do to build out an API and consume an API using Apollo subscriptions. Now before beginning this course, you should be familiar with GraphQL and Apollo. This course is going to build on top of that knowledge that you already have. I hope you'll join me on this journey to learn about event-driven updates with GraphQL and Apollo at Pluralsight.

Creating a Subscription

Introduction

Hey there. Thanks for joining this course, Event Driven Updates with Apollo Subscriptions where we're going to talk about how to build real-time updates into your web applications with WebSockets and subscriptions inside Apollo. All right, now this course is a little bit different in that it's part of a larger ecosystem, and there's several courses that contribute to where we are right now. Now you don't need to go watch those other courses. If you're only interested in subscriptions, don't worry about any of the other stuff. I'm not going to rely on work you have done previously. So don't worry about that. But I'm going to give you just a little insight into kind of where we are through all of this. Now I have a course out there called Building a GraphQL API with Apollo Server. And in that course, we talked a little bit about this company called Globomantics, and

they have a website and a conference that they run. Now as part of that, we built a GraphQL API for the conference data associated with his website. Then Adhithi's course came along, and we got a front end. We got a website using React and Apollo that pulled that data out of GraphQL and displayed it through the front end. And the last piece to this puzzle is Matt's course that allowed us to secure our API and secure the website with JSON tokens and cookies and other things and allow us to sign in and sign out of this website. Now the demo code in the environment that we're going to set up for this course will include code from all three of those courses. Again, if you haven't watched them, don't worry about it, not a problem. I'm not going to assume you know what was built there. I'll kind of show you the stuff you need to know as you go. Now if you have watched them and you've kind of follow it along, great. It'll be perfect for you too. For this course specifically, the problem we're going to try and solve is that on the session page you see we have favorites. And so look right here at AI powered robots. There's a favorite, and there's a 2. And what that 2 means is two people have favorited this session. And so if someone else comes along and favorites the session as well, I want that 2 to become a 3, and I want that to happen automatically without the user having to refresh the page. I just kind of want to keep the user updated as to where favorites are amongst the people who are attending this conference. Now there's two ways to approach this We could pull, which is the normal. That's the way we've kind of always done it in the past is you send a query every second or so that goes and tells me if data has changed, and if data has changed, it'll send back all the new data. And that's fine, that works. But you are now sending a request every second or however long you want to do it, and you've got to stay on top of that, and it creates just a lot of HTTP traffic back and forth from the client to the website. The other option is subscriptions where basically we're going to open a WebSocket, and we're just going to sit and wait. And as things change, the API is going to send new data out to us. And so that's going to be the topic of this course is that WebSocket subscription-based API updating on an event-by-event basis. All right, so in this module, we're going to create a subscription on the API. Now if you only care about the front end, if you're not an API developer, you care about the react side on the front end, you can actually skip this module. You don't have to do this. In this one, we're going to build the API piece. And so if you want to go to the front-end side, skip forward to that one, and then pull down the environment from there, you'll be just fine. All right, but what we're going to do right now is we're going to take our GraphQL schema, and we're going to add a subscription. So far, we've got queries, mutations, and different types, sessions, and speakers, and now we're going to add a subscription type two that schema. And then we're going to implement the resolver for that subscription, just to kind of handle it and send back the WebSocket information. And then we're going to implement a pub/sub model where we're going to subscribe to a certain topic. And when something changes on that topic, we're going to send an update up to the calling application. All right, now that we know what we're doing, let's go get our environment set up.

Our Environment

Let's take just a minute and set up our environment. Now if you go into the course materials on this course, you'll see on module 1 a directory called Starting in the code directory. And inside Starting, we have this code directory that's got an API, it's got an app, it's got a couple other things in it. That's where you want to go. And so I'm going to open up a terminal window in my starting directory, and I'm going to go into my code directory and launch my VS Code editor at this spot. Now when I get in here, notice we've got an app and an API. And I don't care about the app for this module. When we get to the next module where we're talking about the app side of subscriptions, we'll get into that. But for right now, we're going to focus on the API. So I'm going to cd into the API and do an npm install, and that's going to get all of our NPM packages installed and all ready to go. Then we're going to do an npm start, and we'll be up and running. Now if you go to localhost:4000/graphql, you should get this page. And if you get this page, type in query, sessions, all of that stuff, hit Go. If you get data back, you're ready to go. If you don't get data back, don't panic. Either go back and make sure your npm install worked or check here to see what kind of errors you may have. If you see errors there, just drop a note in the discussion forum, and we'll talk about it, and we'll get you taken care of. All right, if you're here, you're ready to do. Let's actually start writing code and adding subscriptions to our schema.

Adding Subscriptions to the Schema

Now the first step we need to take in order to start adding subscriptions into our application is to actually tell GraphQL and tell the end user that we're going to be using subscriptions, like just let them know that that's available. And the way we do that is we add subscriptions into the schema. So real quick, in the demo app, and as we send the last clip, just go grab these demo files and go into the API. And in the schema.js file, this is where our schema lives for our GraphQL application. And you'll see as you kind of look through here, we have some types. We've got SessionInput, Speaker, and then Session. And if we scroll down, now we have a type of query. And this tells the user and tells GraphQL, here's all the queries that you can use, so session query, sessionById, those are the types of queries that we allow. If we scroll down a little further, you see the same thing for mutations. These are how you change data. So we've got createSession, we've got toggleFavorites. Those types of things are all mutations that the end user can have. And so we're going to do the same thing right here below mutations and create a type of subscription. And we're going to put in the subscription type, the query or the subscription that we're allowing. And so we're actually just going to create one called favorites, and it's going to return back— So let's think about this for just a second. So I have a page that has some sessions on it, and it shows how many favorited items that that session has. And so when a favorite updates, I

want to return back the session that was changed and the new count. That's about all the data we need. And so we're going to create a new input type called FavoriteCount, and that's what we're going to put in here. So that's a new type of FavoriteCount, which has two things as part of it. It's got a sessionId, and it's got a count, and that's it. That's the schema. Now we should be able to see that we've got a subscription and a favorites count available in my schema. And actually, we want this to be a capital. There you go. Now real quick, let's actually go out to our API. So we cd into api and do an npm start. I come out to localhost:4000/graphql. If I look at my schema, we should now see our FavoriteCount. And if we scroll down further, we should see our type of subscription. So it's in the subscription now. It's in the schema. And we're ready to go. Now that's great, but it doesn't actually get us anywhere because we see that we support a subscription, but we don't actually support the subscription yet. So in this next clip, we'll get in there, and we'll start actually resolving these subscriptions.

Resolving Subscriptions

Now that we have our subscription all set up, let's actually write the code to resolve our subscriptions. Here in our schema.js file, you see that we have our our Subscription and our FavoriteCount. That's the piece that we now need to build a resolver around to resolve our subscription. So if we go into our resolvers directory, see how we have a whole bunch of different files? We've got mutations and queries and sessions and all that. We talked about breaking resolvers up back in our Building a GraphQL API course, and we're just going to plug into this for a subscription. So we need to create a new file called subscriptions.js, and we just need to do a module.exports. And if we go back to our schema, our subscription is called favorites, and so we need to build a resolver for favorites just like that. Now when you subscribe to favorites, we need to just make note of the fact that somebody has subscribed to this so that we can do something later. We're going to take all the same things we take normally. You have parents, args, context, and info. And then what we want to do is return our pubsub. So the way this works is we have a pubsub, and we're going to subscribe to a specific topic on this subscription model that we have set up. Now we haven't set it up yet. You haven't missed anything. We'll set that up here in just a minute. We're just going to kind of use it here. So basically, the way this works is this pubsub object returns and asyncIterator. And we just listen to this asyncIterator for events that happen. And we have to pass in the queue of updates to our favorites. Now this FAVORITEUPDATES is a constant, so we're just going to do this. You don't have to do that. You could keep it down there and just make it a string. But I prefer to have it defined up above as hey, here's what our topic is that we're subscribing to. Now pubsub comes in on the context. So just like we've done in other places, we're only going to pull in pubsub because that's all we need. Now that's all it is for our subscription. That's the whole thing. Now we do need to include it in our overarching resolver. So we're going to come up here to

resolvers to index and add it down here. Let's add it right under mutations. So then we have our query mutation subscriptions and then all of the other things down below, const subscription = require subscriptions. Excellent. Okay, now we have to add it right down here, and there we go. Now our resolver is there. It's going to use it. But we still need to add something to our server because we've got this pubsub thing that we need to make use of. So let's go down to server.js, and we've got quite a bit that needs to be added here. And so let's actually work through what we need to add as we get through here. The first thing we need to do right here, we've got apollo-server-express that we're pulling in. We also need to pull in this pubsub object from apollo-server, and we're going to add that to the context down below. When I say down below, what I mean is right here after our const app = express, let's just add in right here, const, a new instance of this pubsub object. And as we scroll down and kind of look through the rest of this, we need to add— Find your context. So for me, it's line 67 at this point. It might be slightly different for you. But find the context. And what this is is right here, you've got const server, and we're creating our new Apollo Server. We had the type defs, the resolvers, the data sources. All of that stuff gets added right here. And then down below a little bit, you have our context. And if you remember, we were trying to pull pubsub out of context over here in our subscription resolver. See right there where it says pubsub, it would be context normally. We're just pulling pubsub out. So we need to add pubsub into this context object, and we'll just do it right here. Now if you look up a couple lines, right here on line 69 where it says if req.cookies.token, now for us, for right now, we have no cookies on our subscription. We're going to cover authentication later in a separate module. But for right now, we don't have it. And this will error out if we don't have that there. So I'm just going to add rec in, rec.cookies, just so we clean that up so it won't error out actually when we go do something with it. Now let's look down below here where we've got applyMiddleware and our app.listen. Now app.listen is going to listen on port 4000, and it's going to listen for HTTP traffic. What we now need to add to this is our subscription handler. And the way we're going to do that is we actually need to pull in, up at the top, HTTP. So let's scroll up to the top, require HTTP. We'll save that. And then let's scroll back down to the bottom here, and I'll show you what you need to do to make this all work. So down here at the bottom, we're actually going to, after we apply the middleware, do a const httpServer = that HTTP object we just pulled in, and we're going to create a server. And we're going to pass in our Express app that we've created up above. And then, in this server, the server object that if we scroll up and look it's our Apollo server. In our Apollo Server object, we're going to add this HTTP server, and we're going to install subscription handlers. So server.installSubscriptionHandlers http server. And then instead of app.listen, app would be our Express Server. We're just adding another layer on top of that. We're going to do httpServer.listen. All right, let's start this up, and let's go to localhost 4000. Let's actually call the subscription one time, subscription, favorites, and we're pulling back the sessionId and the count. And we're going to click this Play button. And you see now we get listen. So it's sitting there, and it's just listening

to the API waiting for an update. So if you've gotten this far, hey, things are working. But we're not quite there because it would be kind of nice to actually see it work. So let's take a minute. Let's modify the mutation for favorites so you don't have to have a user, and let's just try it out here on the back end. So if you're just doing the API side, you actually can see this whole thing work

Trying It Out

Now that we've got our subscription working and waiting and listening for an update, let's go and modify the mutation to actually publish to our subscription. So here we are in server.js where we have gone and added all the pieces for the subscription. And I just want to call your attention back to one thing. If we scroll down to our new ApolloServer here on line 50, in our context, we add right here on line 80 pubsub. So we're passing pubsub, this pubsub object, in on the context. And so if we go to mutation.js in resolvers, there's a lot of stuff in here that's been put in here over the last few courses in this path. We're not going to go into what all of these things are, but here's the mutation for creating a session or signing up or signing in and those types of things. We're not going to worry about those. But we're going to scroll all the way down to toggleFavoriteSession. Now this is the mutation that's called when somebody favorites a session in the app, and we're not going to worry about that right now. We just want to use toggleFavoriteSession to go update our subscription. And so here on line 110, if context.user. Again, that's for the app. You've got to be signed in. So I'm not going to worry about that right now. For right now, we're going to come down outside of this if statement, and we're going to put our pubsub publishing in here. After we've tested this all out, we're going to come back. So don't end the clip early. We're going to come back into mutation.js, and we're going to make it right for the next module. But for right now, we're going to reach back into context, and we're going to do a publish. Now over in subscriptions.js, we do an asyncIterator, which is the equivalent of a subscribe, so we subscribe. We're going to copy this FAVORITEUPDATES right here. But now, we're going to publish, and actually, it's context.pubsub.publish to FAVORITEUPDATES. Now FAVORITEUPDATES is a const, so back up here to the top. And you don't, again, need to do that. But just to make everything cleaner, I like it that way. Okay, back down to toggleFavoriteSession. Okay, so we're publishing to this topic, the FAVORITEUPDATES topic, and we're going to return an object. Now this object, if we look at schema.js, is a favorites object, and we're passing back a sessionId and a count. So that's what we're going to do. We're going to pass a favorites object with a sessionId of, let's just pull this, args.sessionId, and a count of let's just do 1 for right now, just for the testing purposes. Then we're going to rerun this and see what happens when I do this. So let's come back over to our browser. Okay, we're listening. If I come to a new tab and I do a mutation, we're doing it for toggleFavoriteSession, and we're toggling sessionId of, let's just do 12. It doesn't really matter because we're not

actually manipulating a session at all. This is just for fun, and we're going to pull back the ID. Now when I run this, if we look back at our code, I don't have a user in context. That'll be in the next module. We're not worried about that yet. So I'm actually going to get an undefined. I'm going to get null back. But I am going to publish onto the context, so my subscription should update. So when I run this, I get null. When I come back over to favorites, I got data back. And I'm just listening. I'm just sitting there listening. If I come over here again, let's change this to 14. Run it again, I still get null. But now I get a second update. So if you've gotten this, your subscription works. You've got subscriptions working in your application, and if that's all you want to do, that's totally cool. But if you're going to carry forth to the next module, then we're going to look at things from the client side, we need to actually make this right because, honestly, I want to pull the right thing. So we're going to cut this, and we're actually going to say, hey, if the user is logged in and they're doing it right, then we're going to publish FAVORITEUPDATES. We're going to pull args.sessionId. But I actually need to get the real count. So the way we're going to get the real count is we're going to pull a list of users from context.dataSources.userDataSource. Now this is the same one. So userDataSource is what's used up above with toggleFavoriteSession. It's going to pull a list of users that have something favorited. And there is a getFavorites method that pulls a sessionId. So again, we're just going to say args.sessionId. We're going to pull that in. Now what this does is it pulls the list of users that have this thing favorited. And so really all I need to return is users.length in my count. That's how many users have this favorited. Now if you wanted to actually send back here's the actual users so you could hover over the number and see the users and all that, that's kind of what this is built for, you could do that. But in this case, I just want the count. That's all we're doing. So I'm going to hit Save there. Now we're done. We've got a subscription up and running, ready to go. Let's summarize things real quick. Then we'll go start looking at it from the client perspective.

Summary

All right, that's it for creating a subscription on a GraphQL API. We've done quite a bit over the course of this. We started by adding the subscription type and talking about what the subscription type was on our schema. We've got queries, we've got mutations, and now we've got subscriptions. Then we moved into creating a resolver. We actually created an entire resolver file, a JS file, for our subscription resolvers and kind of fit it in the overarching application. And then we talked about the pubsub object and the published/subscribe model that goes with subscriptions. And we talked a little bit about the pubsub object that comes with Apollo Server and how you use that to subscribe to a topic and then publish to a topic in our mutation file. We actually set up a mutation that got published to that topic, and then we got to try it out and see how subscriptions would return back in real time. That's creating a subscription. If all you care about is that, there

we go. Now we're going to modify our front-end React app to actually consume those subscriptions using Apollo clients and React.js.

Consuming GraphQL Subscriptions

Introduction

Now that we have a subscription set up on the API side, let's shift our attention and start looking at the client side of Apollo and GraphQL subscriptions. All right, in this module, we're going to focus on consuming a subscription from the API. The first thing we're going to do is we're going to take a tour of this application that we have set up. Now this is part of a learning path. So there's several courses that have come before this, and we're just going to take the output of those courses and use that in this module. So Adhithi did a course on Client Side Apollo and GraphQL, and that's the demo that we're going to use for this. Now you don't have to go watch that course. If you haven't watched it already, don't worry about it. I'm going to take you on a tour of the application. I'm going to show you what it does. And then we'll use that to build out our subscriptions. The first thing we're going to do is we're just going to create a brand new page and we're going to view a subscription. We're just going to create something blank. and we're just going to hook up to a subscription, and we're going to run it that way. But then we're actually going to go into the business case that we're trying to deal with, and we're going to use this SubscribeToMore method that's going to just pull updates. So we have a query that goes and it pulls favorites. Then we're going to add a subscription on top of that and just update automatically anytime anybody else clicks Favorites. All right, with that being said, let's go take a quick tour of this application that we're going to use in this module.

A Quick Tour of the Application

Now I told you in module 1 that you didn't have to do the API side of this course. You can just jump in and use just the client side of the course just to learn the React side. And that's totally cool, and that's the part where I'm at. So what I'm going to do now is give you a quick tour of this application, just the client side, so you're accustomed to where everything's at, and then we're going to jump in and we're going to start building out the client side of this subscription model. All right, in the code directory of the course materials, you'll see two folders, a starting folder and an ending folder. If you pop open the starting folder, don't go to ending. Everything is written there. Don't go there. Start in starting. We're going to open up a code window, and you've got an API and an app. Now the API is all of the stuff we were working on in the last module. We're not going to worry

about that right now, but we do have to have it up and running. So let's open up a terminal window, cd into the API, and they do an npm install just to get everything we need for that and then our npm start. And we'll see down here at the bottom, GraphQL running on port 4000. That's all set. Now let's open up another terminal window and go into the app directory. Same thing, npm install, and then we'll run this with npm start. Now as soon as you run this, it's going to pop open a window that looks just like this. And this is the website we're going to use throughout this. So if you go to the Conference tab and you view the sessions, this is where we want to be. We've got a list of sessions, and there's no favorites. I've been talking about favorites up till now. And if you are signed into this application, it allows you to favorite sessions. So what you need to do now is come up to the right corner to sign in and actually sign up. Create an email address and a password. I know I'm not using my email address. We can even do validation. All right, now I'm signed in. I've signed up. View Sessions. Now I get these favorites here, and I can click Favorite, and it makes it yellow. That's the application as it stands right now. What we want to do is when somebody clicks Favorite here, we want it to not only update here where it says 0, but we also want everyone else to get an update that lets us know how many people have that selected. Okay, if we run back over to the code, I'm going to show you just a couple of things real quick. In the app directory, in src, you'll see we've got our App.jsx and pages, our conference pages, sessions, speakers. All of that's lined out right here. There's also one other thing that you'll want to know about, which is the context directory that has our Apollo provider. Those are the areas of the application we're going to be working in. Now if you get lost and you're like I don't know what the Apollo provider is or I don't know the sessions and pages and all of that kind of stuff, don't worry. There is the Apollo Client course that built all of this, and you can go back and you can watch that course and you can kind of situate yourself and understand where everything is at. Now one other thing I want to point out. If you didn't do the first module, you can come up here into the API directory, and all the data for this course sits in this data directory. And if I go to users, you'll see there's my user right there, and I have two favorites, and Matt has a favorite left over from his course as well. So that's the app. If you're here on this spot, you're ready to start building a subscription that's going to pull in FavoriteCount every time somebody clicks on the favorite list. So now it's time to actually get to work. Let's go build the client that's going to pull those subscriptions in.

Consuming Subscriptions

Now that we have an idea of kind of what this application looks like, let's build a page that's going to consume this favorites count subscription. All right, so here in our VS Code window, let's right-click on conference under pages, conference, create a new file called FavoriteCount.jsx. And just like all of the other pages that we have here, we're just going to pull in the normal import * as

React and useQuery, gql, all of that kind of stuff. And then we'll build our page out below this. Now useQuery right here on line 2, useQuery is the function you use if you're going to call a GraphQL query. If you look at sessions.jsx, you'll see we have useQuery and useMutation. And that mutation is what's used to add a new session. Well, you guessed right if you guessed that for a subscription, there's a useSubscription. And that's what we're going to use to wire up to the subscription we created back in the last module. So right here on line 6, we're going to create our subscriptionQuery. If you did module 1 with us, you'll remember we had this subscription favorites, sessionId, countId. That's what we want to use for our query. And so we're going to create our FAVORITES query = gql, and we'll add that subscription there. Now we name it favorites just like that. Okay, now we have our query, we need to create something that's going to call this query and wait and look for updates. So let's create our function, our React function that's going to render the page and show us the results of our subscription. Now useSubscription, the method we're going to use to call the GraphQL API for the subscription, returns three items on an object. It's got loading, which tells us the status of it, error, which is going to show us errors, and then data, which is the thing that's going to have the return results in it. And we're going to call our FAVORITES query, just the one that's right up above. So if we're loading, we're going to return loading. If there's an error, we're going to return the error. Otherwise, we're going to return the data associated with this new favorited item. So if we're not loading, we're going to pull the favorites item off of data and give us back the sessionId. So basically, if we go back and look at the screen, every time somebody clicks this button right here, we want to show, hey, we have a new favorite item and the ID of the session that was just clicked. That's what we're going for. Now one more thing we want to do is we've created a page. Now we just have to add it. So let's go down to our App.jsx. And right here under Conference, we're going to pull in our FavoriteCount. So that's the page we just created. And then down here in our switch statement, let's just add right here at the top a favorites. Done. So in theory, we should now be able to go to Favorites and have everything work. But it doesn't. And everything blows up. And I get cannot read property sessionId of null. That's because things aren't going back. What's actually interesting is if you inspect the network, let's actually refresh this page so we see everything. And what you'll see is this cancelled GraphQL call right here. And this, if we scroll down far enough, this is our subscription call, and it got canceled. It blew up. It said no, I don't know how to do this. And the reason for that is because it doesn't yet know how to do WebSockets. And so, in this next clip, let's actually look into the Apollo provider and build out the WebSocket handler for our subscriptions.

Expanding Link for Subscriptions

In order to make subscriptions work, we've got to go into our Apollo provider and expand the links section in order to include WebSockets and subscriptions. All

right, if we go into our Apollo provider file in context is the directory. If you look down through here, and you kind of just look at what this is doing, so there's our client. We're creating a new Apollo client. And here in the link, we have a new HttpLink, and that's what is used to serve up our queries and our mutations. But for subscriptions, we're doing WebSockets. And so we've got to add a piece to this in order to enable WebSocket communication from the Apollo client back to the Apollo server. And so what we want to do is actually take this out. Let's just pull that and come up here to the top and just do that for us. And what we're going to add to this is, that's our HttpLink, now we actually need to configure a WebSocketLink as well. And then it's going to choose between the WebSocketLink or the HttpLink as to what Apollo is going to use for the query. So we're going to create a new WebSocketLink, which we don't have yet. We've got to pull that in. And so right here, we're going to pull in WebSocketLink from apollo/client/link/ws. All right, so we've got that, and that's ready. Now we've got to actually create this new WebSocketLink, and it's very similar to HttpLink. As a matter of fact, let's just copy this and drop that in just like that. Now the URI that we're going to use is actually ws:// :4000/graphql, and we'll save that. Now we have to do something to enable WebSocket and HttpLink to understand which thing it's supposed to be using at any given time. And that's done with an object called split. It just comes part of Apollo client. It's just there. We just pull that in. If we scroll down to our link section, this is the link section, there's a little bit of boilerplate code that we have to use. And it's the same everywhere. So basically, what's happening is we're going to split, and we're going to look at the query. And if the query is an operation definition and it's a subscription, we're going to use the WebSocketLink. If it's not a subscription, we're going to use the HttpLink. So let's actually even, let's just pull this out even more and just do that. Let's just keep it clean where we can do that and come up here, and let's just paste this up here. Now there's two more things we have to do. So right here getMainDefinition, which we added in split. We don't know what that is, and we have to pull that in up here as well. And all getMainDefinition does, if you guys kind of look at that, is it just tells you what kind of operation the query is. This is a little utility that spits some stuff out. So that comes from apollo/client/utilities. Let's just pull that in. Now there's one other thing we need to do, which is this WebSocket piece actually comes from a new NPM package. So let's npm install subscription-transport-ws. Let's pull that in, and now it should just work. So let's go back over to our previous page, and now we get a loading screen. It's just sitting there. It's just waiting. Let's duplicate this tab. Well actually, let's look at this just like this. So if I go to Conference, Sessions, and I click on Automation: A Deep Dive into Jenkins, I got my new favorite, 84375. That'll be the number associated with this. If I go favorite this one down here, it updates. Look at that. That just works. There's your subscription. Subscription is working. We have basically just opened up a WebSocket back to the API, and we're just sitting there and waiting for an update. And when that update comes, we display new data, and it just cycles through and sits there and waits and holds this open until it's done. Now that's only half the battle. What I really want is this number next to

Favorite to update whenever somebody else or me clicks on the favorite. So I want two windows open, both on this same conference session page. And as I click Favorites, I want the numbers to change. Now that we've got the subscription working, we can now use that same mentality, the same code, to add a piece to our query that enables us to subscribe to more updates and sit and listen to update a query.

SubscribeToMore

Now that we have the subscription in place and everything seems to be working, let's look at how to actually update queries that we've already run by using this function, SubscribeToMore. Now if you come back to FavoriteCount and look right here in the middle, we've got this favorites query that we have. Let's copy that, and we're going to use that over on our other page. So if we go to Sessions.jsx, you see there's our SESSIONS query. There's our SESSIONS_BY_ID query, CREATE_SESSION, TOGGLE_FAVORITE session, all of that. Let's come right down here and add our FAVORITES query. And we're going to use that in order to update our sessions list. Now if we scroll way down to our SessionList query, so for me, after doing that paste it's on 150. We're going to add our subscription in here to automatically update changes to session. And this useQuery comes with another method called subscribeToMore. All subscribeToMore is is a function that we're going to call. And subscribeToMore takes an object. That object needs two things. First, you have the document or the query we're going to use or the subscription we're going to use to do our updating, and that's going to be this FAVORITES query we just copied in here. And then the next thing it takes is updateQuery. This is going to be a method that's called whenever new data comes back, and this is what we use to update the data in our sessions list. So they're going to pass us the previous object. And then on top of that, they're going to pass us another object, and we're going to pull from that subscription data. So we've got the previous object, and we've got the data coming out of the subscription. And we want to return a new object with a combination of the previous data plus the new subscription data. So first and foremost, if there's no data in this update, then I've got nothing to change. So we're just going to return that, just like that. Otherwise, let's take our new data, so this new object that we're going to recreate, and let's take a copy of that. And then what we want to do is say, hey, our newData.sessions is going to be, so this new array of sessions, is going to be the old array of sessions, but then updated with the data that came back. And if you recall from the previous clip, we're sending back the sessionId and the new favorite count. So we want to loop over the previous sessions and update the count wherever appropriate. So first things first, let's take a copy of the old session and then say, hey, if this new session equals subscriptionData.data.favorites, which is what the query, so let's scroll back up. So favorites is the is the subscription.sessionId. Then, let's update the favoriteCount and then return s and then return newData. That should get us there. But let's

pop open our command line. We see we have restarted. All right, so let's refresh these two pages. Now when I click here, I should see the same one update in the other browser, and it did. And if I scroll down, there we go. Now we are sitting and waiting for a favorite to change. And when it does, so in the browser over here on the right, we're not doing anything over there. It's just sitting and waiting. And when this subscription changes, it goes back to the server, makes the change, and then updates both browsers. Because both browsers have a subscription open, it'll update both of them and let them know what the new numbers are. Excellent. All right, let's summarize this real quick, and then we're going to get into some authentication.

Summary

All right, we've done it. We have successfully consumed a subscription from an API. We started out with an application that was already in place. So we used the client side application from the Client Side GraphQL course, and then we did a couple of things. We created a page to view subscriptions. We just created a standalone page with view subscriptions, and it kind of did its own thing. But then we did a real use case with subscriptions using the SubscribeToMore where we had a query, we had a list of sessions already, but then when somebody changed that session, it automatically updated through WebSockets and sent an update out to all the browsers that said, hey, something has changed. So that's it. You've got the API side where we've created a subscription. You've got the app side where we're consuming a subscription. But there's a couple of pieces that are missing that we haven't yet talked about, and the first piece that we're going to talk about in this next module is how to secure a subscription, so actually use the login and password that was built in the Securing Applications course and use that in our subscription.

Securing GraphQL Subscriptions

Introduction

All right, now that we've got the client and the server side working with our GraphQL subscriptions, let's take a minute and talk about how to secure these subscriptions. All right, in this module, we're going to focus on securing GraphQL subscriptions. Now we're going to use some existing security code. There is a whole GraphQL security course built by Matt that already exists in the GraphQL Apollo path. Subscriptions are a little bit different, and they add another layer on top of that. So instead of rewriting the entire security content, I'm just going to leverage what Mat Warger already built, and then I'm going to take a step on top of that. Now in order to do security with subscriptions, we have to take a slight

detour for a minute and talk about the subscription lifecycle and the on connect and on disconnect aspects of a subscription from the server side. Now we're going to use tokens to authenticate users, and I'm going to show you how to pull that token out and authenticate a user using that token. And while we're talking about lifecycles, I'm also going to real quick talk about some auto reconnect options we have on the Apollo client side. All right, with all that being said, let's jump right in and talk about the subscription lifecycle.

The Subscription Lifecycle

Let's take just a couple of minutes to talk about some lifecycle events that happen when you connect to and disconnect from a subscription. So here we are back in our API code. And right here is our server.js file with all of the stuff in it that that we've done over the last couple of courses. And so if we scroll down to right here line 50 where we are creating our new ApolloServer. Now so far, here we've got typeDefs, resolvers, dataSources, schemaDirectives, validationRules, context. ApolloServer takes a whole lot of stuff as config items for this ApolloServer object. But we're going to add another one to it right now. We're going to add something called subscriptions. And subscriptions takes an object, just like schema directives or context or any of these other things. So subscriptions takes an object, and we're going to pass two items on this object into subscriptions. The first one is going to be onDisconnect. And what onDisconnect is going to do is fire every time a subscription is disconnected. We'll just log that out. Now normally in here you'd put some cleanup code if you needed to or invalidate a token or whatever makes sense for your specific thing. But there's an onDisconnect method that gets called here. And you guessed it, there's also an onConnect. Same thing. This is going to take a function, and we're just going to console.log out Connected. And I mean there's a couple of things we can pull in here. There's some params, there's the WebSocket, that we're not going to do anything with right now, but those are there Okay, so we've got onDisconnect and onConnect. If we open this up, our terminal window, when I refresh conference sessions, there's my connected, and it executed a whole bunch of stuff. If I close this browser, there's my disconnecting. So I got my connected. I got my disconnecting. Everything worked exactly the way I expected it to for my two lifecycle events. Now that we have our lifecycle events in place, we can use that onConnect method to secure our subscription.

Securing Subscriptions

Now that we have our onConnect set up, we can leverage that to secure our subscriptions. So in order to turn this into being able to use the cookie-based authentication that we're using for the whole application, we need to hook

into this WebSocket right here in onConnect. So real quick, let's do a console.log and add to this Connected. Let's do the WebSocket.upgradeReq.headers and just pull the cookies out of the headers, and we'll save this. Now there's one thing we need to change because when we do a login in our API, so let's go into resolvers. Let's look at the mutation resolver. Right here in signIn, if we scroll down to signIn, when we create this cookie, we're creating it for httpOnly, and we want that as false so that we're sending it over WebSocket traffic, as well as HTTP traffic. Okay, let's come back to our server, and we're spinning out our headers.cookie. Okay, so let's open up our terminal window. And when we open our localhost, look right here. We've got a token. And notice what it says though. It says token equals and the cookie. So we're pulling the cookie directly out of the headers on the WebSocket already based upon everything that we have. And so we need to use this token in order to go through the process of authenticating the user. And if you scroll down here in context, notice on line 79, we do an auth.verifyToken with the token in the cookies, which is awesome. We want to replicate that process up above. And so if we come back up here, let's actually just do that. Let's do a let user and then pull this cookie out. Now we don't want the token equals. That's a little weird. We don't want that. We just want the token. So we have to strip that off. So let's just do a .split to pull that token out, and it'll be the second item in the array. The first item is going to be what's before token, which is nothing. And the second item is going to be just this token. Now let's go get our user. So we're going to use auth.verifyToken, and we're going to pass in the cookie, and we need an equals sign. Oh, we already created the user up above. Let's get rid of that. We don't need that. Let's save that. All right, now when we reconnect and refresh, I've got my user. Connected, my user, my role, all of that. Now that doesn't necessarily buy me a whole lot because it's in my onConnect, but it's not available to the larger application yet. So what we actually can do out of onConnect is return an object, and that becomes available to us down below. So if we come down to context, instead of just req and res, we can add to that connection. Connection is another variable available to us in this context object that gets passed into the context function. And what we can do now is leverage that to add the user. So right here, after let user = null, we're just going to say, hey, if connection and connection.context, so if we have a context on connection, we're going to say user = connection.context.user. And where that got added to the context was up here where we returned user. That adds it to the connection context. So the onConnect creates a connection object. The context is what we return out of onConnect. And user is an object on that context. And so now, we have our user available to us. So if we save this, see we'll restart. And now if I go to my subscriptions, resolver, now in context, I not only have pubsub, I also have user. And I can say console.log. And now when I run this, I get nothing. I get connected with a query, but nothing's resolved yet. Now we go. When we go to this page, I'm actually creating a subscription, and I'm listening on a subscription. And now look, my subscription resolver has access to my user object, and I can use it there. So that's the process. We just leveraged all the work that Matt had already done in the subscriptions course. But building on top of

that, we now, using the onConnect method of the subscription lifecycle, we are pulling a cookie out of the headers, and we are verifying that cookie and then adding it back to the context that the ApolloServer has that we can leverage using our subscription resolver. So there you go. That part's done, but I kept having to refresh. Every time I changed something here, I'd have to go refresh the website in order to make it see the changes, and I don't necessarily want to deal with that. What I really want to do is auto reconnect. Every time there's a hiccup or a challenge between the back-end server and the front-end server, I want you auto reconnect that dropped subscription. So let's take a minute and let's do that.

Auto-reconnect Dropped Subscriptions

Now let's just take a minute since we have our subscriptions working and we've got our auth working and we've got all of that there. Let's look a reconnecting dropped subscriptions from the client side just for a minute. This is really quick, but it's very important. All right, if we come in here to our server code that's running, and I just do an rs just to restart the the API. What happens when that happens is it drops the connection, our WebSocket connection, back to our client. And so now when I click on Favorite, notice it doesn't update. Now once it redraws the page, it'll connect automatically. And so now we can even see we've written some resolvers back, and it did reconnect eventually, and now it's back. So that does work. But that first one, if I restart again, that first one doesn't work. And the reason for that is because that connection is gone. It automatically just drops everything. So we want to fix that. We want it to detect a dropped connection and then automatically reconnect. And the way we do that is if we come back into our app code. And what we're going to do is we're going to come into our app, and we're going to go to our ApolloProvider. And in our WebSocketLink, this actually takes options. And one of these options is reconnect. We're just going to say, hey, automatically reconnect when something fails. And now if I restart this, you actually see immediately that reconnect. And there it is, just like that. So if I do a restart, immediately it's reconnected, and now it works. All right, that's it for authenticating our subscriptions. Let's summarize real quick, and then we'll go over to filtering our subscriptions.

Summary

We have worked through securing a GraphQL subscription in this module. Now we didn't do all the work because there's a whole lot of work they're involved in securing a GraphQL API, and I didn't want to redo a lot of that work that Matt has already done in the Securing a GraphQL API course. So we took what we had there, and then we had a conversation about the subscription lifecycle because in the onConnect lifecycle event, we needed to pull the token out in order to

authenticate the user. So we had to start there at the lifecycle event. And then we, like I said, pulled a token out and authenticated a user through a token in the onConnect. We talked about the context and how we add that user to the context so that it's available to us over in our resolvers. And then since we were talking about lifecycle events in this module, we just tacked on auto reconnecting on the client side. So when the server side drops a connection, the client side will auto reconnect that subscription, so you never miss a subscription event. All right, now that that's done, there's one piece left to GraphQL subscriptions, and that's filtering a GraphQL subscription.

Filtering Subscriptions

Introduction

In this module, we're going to tackle the process of adding filters to our subscriptions. Now this process of filtering subscriptions has a couple of steps involved with it, the first one being we need to adjust the schema. And this is pretty straightforward part where we've got to add something to the schema to let us know that we're allowing a variable to be passed into filter on. Then we've got to resolve with filters, and this is a little bit more complicated because of the way the subscription works. It's not all at once. It's going to happen over time, over and over again. And the last piece is adding variables to the client. We're going to modify the subscription query to include variable so that we can filter there on that side. All right, first things first. Let's get in and adjust our schema.

Adjusting the Schema

All right, the first step in this process is adjusting the schema on our API. So if we come into the schema.js file in our API code, this is the starting point to let the server and the client know what's allowed on our subscription. So if you look right here at line 94, you see our type of subscription, and we have a favorites subscription that gives back a FavoriteCount. So in some cases, we may only want to filter by a specific ID, and that's it. And so that's kind of the idea that we want to go with. We want to add right here a sessionId that's an ID. And so the idea here is, if you pass in a sessionId, we're going to filter and only send you updates for that specific session. So if you are on a specific session page, you only want a subscription for that one page, you don't need it for everything else. And so that's the idea that we've got going on right here. So this change, just like that, should be everything we need in order to get started marching down this path. Unfortunately, the next step is a little bit more complicated because we need to start resolving filters based upon whether you send nothing at all or you send a specific ID.

Resolving Filters

All right, now that we've got a schema in place, let's start resolving our filters on our subscription. So if we come into our subscription resolver, right now it's pretty simple. We've got our our subscribe on our favorites that's returning our asyncIterator on pubsub. And the trick with a subscription is that this asyncIterator returns every time something changes. And so we can't filter right here in subscribe because all that subscribe does is return an asyncIterator. It doesn't do any filtering there at all. So we've got to add a layer on top of this to get our filter done. And the way we're going to do this is we're going to add something up here on top called withFilter that we're going to pull out of ApolloServer. And what withFilter does is allow us to add a layer on top of this subscribe object. And so what we do is we add that right here, and this is a method that takes two parameters. The first parameter is going to be our function that returns our asyncIterator. That's the normal thing that we've already got in place. So we're just going to leave that they're. The second thing it takes is a second method that allows us to filter any event that happens from this asyncIterator, and it's going to pass us two things. It's going to pass us the payload. That's the thing that is returned from the asyncIterator. And then it's going to pass us the variables associated with the subscription. So when we do a subscribe, we're going to pass some variables in. Well, it's going to hand us those and let us know what's going on. Now in the last clip, we set up some rules. We said that if we pass nothing, we want all the events to come back. And if we pass an ID, then we want to just get events about that ID. Well, what this method does is it returns true or false. It's a filter. So either we allow the item to come through or we don't. And so real quick, if there's no sessionId on variables, then we're going to return true because if we didn't pass a sessionId in on variables, then we want everything. So we just always want to take the difference. Now, however, if there is something on variables, if that equals was coming in on the payload, then we want to filter that. So let's look at the payload and favorites— The reason why we have favorites, if we look back at our schema, that's the name of the subscription. So you've got the payload.favorites.sessionId, and we're going to return whether or not those are the same or not. Now there's a lot that could go into this. So if you want to return an array of IDs, then you would return a contains, or you would look in the array to see if they overlap or things like that. All of that code would go in here depending on what you are trying to accomplish with your specific use case. But in this case, if we pass in a sessionId, then we're just going to check if that sessionId matches, and then we're going to return true or false. Let's save that, and that's it. We've got that up and running. So the next step is going to be adding variables to the client side so that we can filter out these events.

Adding Variables to the Client

All right, let's round this out by adding variables on the client side to filter out subscription events. If we come out to the app side, and we go into src, pages, conference, and we go to the Sessions.jsx page, now this is where everything's happening in our sessions list. And if we scroll down to our subscription, const FAVORITES = subscription favorites, all of that, this is where we're going to add our variables. And the first thing we want to do is put in here the fact that this takes variables at all. So $sessionId is an ID, and that lets the subscription know, hey, we're going to pass a variable called sessionId sometimes, not always, but sometimes. If it was going to be always, you put the exclamation point. All right, now in favorites, we're going to say, hey, sessionId is going to be equal to this variable that we created right up above. All right, now that gives us the variable sessionId that's going to be passed to favorites when we run this. All right, if we scroll down to our subscribeToMore, let's actually just put this on our subscribeToMore down here on 157. So when you do a subscription, we're passing the document, which is favorites. On top of that, we're going to pass in the variables. Now for us for this, so we can see this work, what we're going to pass in to sessionId is let's just take whatever's in data, let's just take the second item in the array and pass that through. And the reason why I'm doing this is we could hardcode something, but I want to actually take an item in the list and say, hey, this one, we're going to capture updates on. Everything else we're going to ignore, but this one, we're going to capture updates on. Now when I look at the sessions list, what I expect to see is when I click on this second item, it's working because I'm filtering by that second item. When I click on this first item, I get nothing. Nothing happens because I'm very specifically only looking for subscriptions around this second item in the list. Now if I come back in here and I take this sessionId out and I save that and it refreshes this page, now everything will work because we said, hey, if we're not passing a sessionId, everything should work, and everything works. But if I say, hey, I only want to take this first item in the list or the first array item, which is the second item in the list, that one will work, but none of the others will update. All right, that's it. That's filtering and passing variables from the client side back to the server side. It works the same way as queries and mutations and that stuff does. It's not very different. I just wanted to walk through the process with you so that you could see how that works. All right, so let's do a summary, and then that's it. We're done.

Summary

All right, that's it for filtering subscriptions. Now there were three steps that we had to go through. The first step was adjusting the schema, so we had to add something to the schema to let it know that we were going to be passing variables back and forth on this subscription. Then the complicated part of this module was adding variables to the resolver itself and using that withFilter method that allowed us to check every time that asyncIterator returned to make

sure the thing that it returned with something we wanted. And then we just made sure it worked by adding some variables into our client in that subscribeToMore method to make sure that we got back what we needed and only the second item in the array would update. Nothing else would update. And so that made sure that this resolver worked, and now we knew how we wanted to move forward. Okay, so that's it. That's the course. We've done everything. We built the API side. We built the app side. We did some authentication. We added some variables and filtered. That's the whole gamut of what it takes to build a subscription in GraphQL and Apollo.